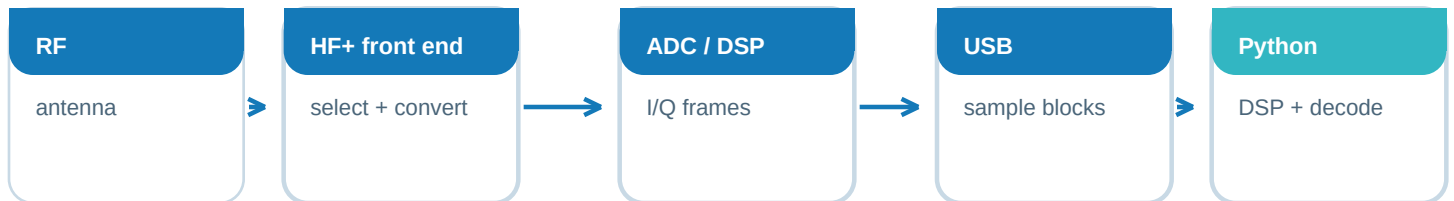


I/Q from antenna to Python

A technical companion to the primer: bandwidth, aliases, filters, and real sample math.

The one-sentence model

The Airspy HF+ converts a narrow slice of radio spectrum into timed I/Q pairs. On your Mac, Python treats those pairs as a complex vector stream, then shifts, filters, downsamples, and demodulates it.



Division of labor

IN THE AIRSPY HF+

RF filtering and tuning reject unwanted energy before digitization. The receiver mixes the desired slice to a low intermediate/baseband region, digitizes it, and performs device-side processing before USB.

ON YOUR MAC

Python receives blocks of signed samples. Convert them to complex values, tune digitally, apply channel filters, decimate, and choose an AM, FM, or digital demodulator.

Two numbers per sample

$$x[n] = I[n] + j Q[n]$$

I and Q are not left and right audio channels. They are perpendicular projections of the same RF signal.

Scope note

Published HF+ family material commonly describes an 18-bit ADC path, 768 kS/s output and roughly 660 kHz of usable spectrum; exact bands, firmware, gain behavior, and driver sample formats vary by HF+ model and software. Treat the device settings and driver documentation as the final authority.

Sample rate sets the digital canvas

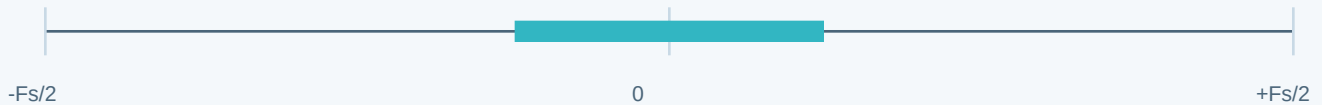
Frequency in DSP is always measured relative to the sample rate F_s .

Nyquist in one picture

With complex I/Q, the digital spectrum spans $-F_s/2$ through $+F_s/2$.

At $F_s = 768$ kS/s, the formal edge is plus/minus 384 kHz. Real receivers and decimators need transition room near the edges, so the useful span is smaller than F_s .

usable complex spectrum



Worked Airspy-style example

DEVICE OUTPUT

$F_s = 768,000$ complex samples/s.
Nominal Nyquist is ± 384 kHz
around the selected center
frequency.

CHANNEL WANTED

A 10 kHz AM station at +95 kHz
offset is comfortably inside the
passband.

AFTER DECIMATION

If filtered to 12 kHz, decimate by 16:
new $F_s = 48$ kS/s. That is plenty for
AM audio.

Rules of thumb

Complex bandwidth

A complex sample rate F_s carries about F_s Hz total spectrum, centered at zero.

Guard bands

Do not park a wanted channel on the device passband edge; filters need room to roll off.

Data rate

768 k complex I/Q pairs/s at 16 bits per component is about 3.1 MB/s before USB framing.

What folds when sampling is too slow

Sampling cannot tell apart tones separated by whole multiples of F_s .

Alias frequency

$$f_{\text{alias}} = ((f + F_s/2) \bmod F_s) - F_s/2$$

Bring a frequency into the interval $[-F_s/2, +F_s/2]$. Anything outside wraps back in. The wrapped signal is indistinguishable after sampling.

Example: $F_s = 48 \text{ kS/s}$

12 kHz input

appears at +12 kHz

safe

36 kHz input

appears at -12 kHz

aliased

60 kHz input

appears at +12 kHz

aliased again

Where protection happens

BEFORE THE ADC: HARDWARE

The receiver front end, preselector, and analog filtering reduce very strong out-of-band signals before they can fold into the ADC. No later software can undo an analog alias already created.

BEFORE DECIMATION: SOFTWARE

After tuning a channel, use a low-pass filter before lowering F_s . The filter removes content that would otherwise fold into your new, smaller Nyquist interval.

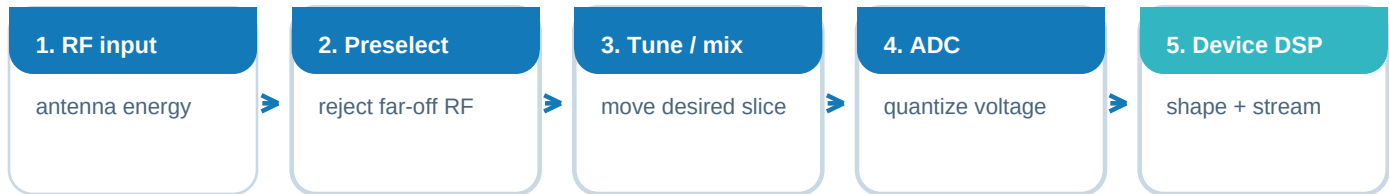
Key distinction

Hardware anti-alias filtering protects the ADC. Software anti-alias filtering protects each decimation step.

What happens before the Mac sees I/Q

The hardware makes the first bandwidth and dynamic-range decisions.

4 / 8



Hardware decisions that matter

CENTER FREQUENCY

Choosing a center frequency determines where your target sits in the output spectrum. A 7.100 MHz signal with center 7.050 MHz arrives around +50 kHz baseband.

GAIN AND OVERLOAD

Gain controls the trade between weak-signal resolution and headroom. A strong nearby broadcaster can overload stages even if your desired station is quiet.

Practical setup example

Target Listen to an AM signal at 7.100 MHz.

Airspy tune Set center near 7.050 MHz. Target appears near +50 kHz.

Why offset? Avoid the exact center/DC region and leave room for a software channel filter.

USB frames The driver sends consecutive blocks of I/Q values; timing continuity between blocks matters.

Do not expect Python to fix front-end overload.

If clipping or intermodulation happens in hardware, reduce gain, improve filtering, or change antenna/front-end conditions.

From driver bytes to complex vectors

The exact driver API differs; the DSP pattern stays the same.

Representative block-processing shape

```
# iq_i16: interleaved signed I, Q samples from your SDR driver
iq = iq_i16[0::2].astype('float32') + 1j * iq_i16[1::2].astype('float32')
iq /= 32768.0 # normalize if the driver uses int16
n = np.arange(iq.size)
iq *= np.exp(-2j * np.pi * offset_hz * n / fs) # digital tune
channel = scipy.signal.lfilter(taps, 1.0, iq)
```

Important engineering details

BLOCK BOUNDARIES

Keep filter state and oscillator phase across blocks. Restarting either at every block creates clicks, spectral splatter, or phase jumps.

SAMPLE FORMAT

Confirm whether data are int16, float32, interleaved IQ, or separate I/Q arrays. Also confirm the sign convention for Q.

TIMING

Process fast enough to consume USB blocks. If you drop blocks, time and phase continuity are broken.

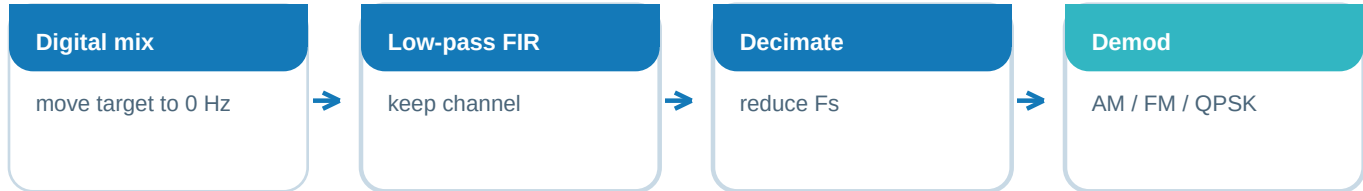
Minimal macOS environment

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install numpy scipy matplotlib
# Then use the Aircspy-compatible driver/library for sample capture.
# Keep capture separate from DSP: queue blocks to a worker thread.
```

Choose the channel, then lower the rate

Filter first; decimate second. That order prevents software aliasing.

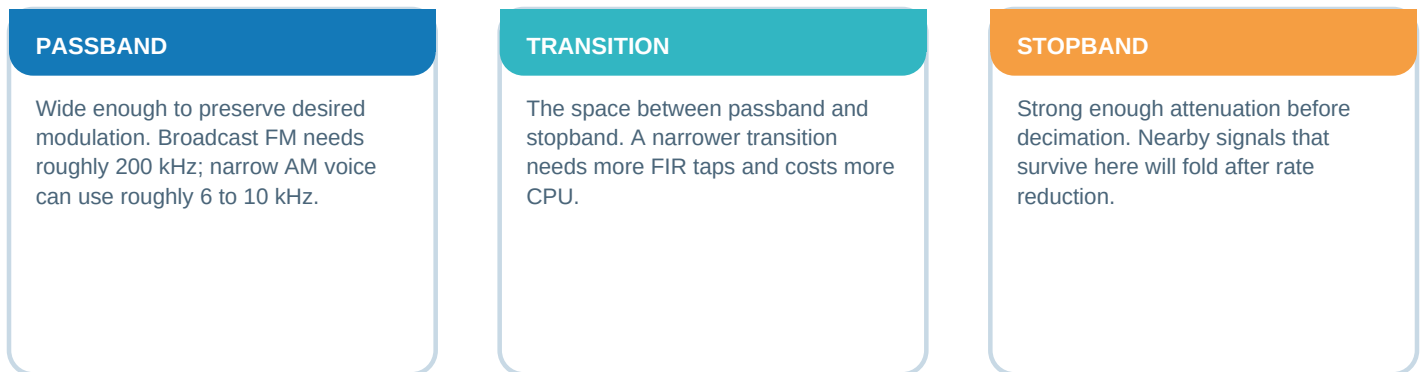
A common channelizer



FIR design example for a 10 kHz AM channel

```
fs = 768_000; cutoff = 6_000; decim = 16
taps = scipy.signal.firwin(numtaps=161, cutoff=cutoff, fs=fs, window='hann')
baseband = scipy.signal.lfilter(taps, 1, tuned_iq)
baseband = baseband[::decim]      # only safe after the low-pass filter
fs_audio_path = fs / decim        # 48,000 samples/s
```

How to choose filter settings



Filter visual intuition

A filter is a gate in frequency, not time. Digital tuning places the wanted station at the gate center. Then decimation throws away samples only after unwanted frequencies have been suppressed.

AM and FM are different motions of the same vector

The numbers below are simplified, normalized samples at equally spaced times.

AM: magnitude carries the message

Suppose a carrier is already tuned to zero. Its angle stays nearly fixed while its radius grows and shrinks with the audio envelope.

n	I	Q	magnitude
0	+0.80	+0.00	0.80
1	+1.00	+0.00	1.00
2	+0.55	+0.00	0.55
3	+0.80	+0.00	0.80

AM estimate

```
audio[n] = abs(I + jQ)
```

Then remove DC and low-pass.

FM: phase change carries the message

FM keeps amplitude roughly constant. The vector advances by a different angle each sample; larger angle steps mean higher instantaneous frequency.

n	I	Q	angle
0	+1.00	+0.00	0 deg
1	+0.92	+0.38	22 deg
2	+0.42	+0.91	65 deg
3	-0.30	+0.95	108 deg

FM discriminator

```
angle(x[n] * conj(x[n-1]))
```

This reports phase step per sample.

```
# AM (after channel filtering): envelope = np.abs(x)
# FM: phase increment using adjacent complex samples
fm = np.angle(x[1:] * np.conj(x[:-1])) * fs / (2 * np.pi)
# Low-pass / deemphasize / resample audio after demodulation
```

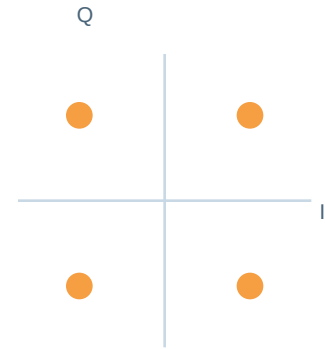
QPSK points, timing, and a real workflow

Digital modes need carrier recovery, timing recovery, and decisions in addition to filtering.

QPSK: bits select one of four points

After pulse shaping and synchronization, symbols cluster around four diagonal points. One common mapping uses the signs of I and Q to represent two bits.

bits	ideal I	ideal Q	complex point
00	+0.707	+0.707	$+0.707 + j0.707$
01	-0.707	+0.707	$-0.707 + j0.707$
11	-0.707	-0.707	$-0.707 - j0.707$
10	+0.707	-0.707	$+0.707 - j0.707$



Why raw samples are not yet symbols

Carrier offset

Constellation rotates over time. Correct with a carrier loop or estimated phase/frequency.

Symbol timing

You must sample at the best instant. Use timing recovery after matched filtering.

Pulse shape

Transmitters commonly use root-raised-cosine shaping; use a matched filter to improve SNR.

Decisions

After synchronization, choose the nearest constellation point, then map points back to bits.

Suggested end-to-end experiment on your Mac

1. Capture a short complex I/Q file at a known center frequency and sample rate.
2. Plot a spectrum; digitally tune the target to 0 Hz; inspect again.
3. Filter and decimate conservatively; validate no adjacent energy folds in.
4. Try AM or FM first; move to QPSK only when spectrum, clock, and carrier are understood.

Takeaway

The HF+ digitizes a slice; Python makes the final channel, rate, and demodulation choices.